

AI Attack

task 1 对抗样本攻击

运行实验文件中的 model.py 来测试环境是否配置成功，结果如下：

```
C:\Users\95441\AppData\Local\Programs\Python\Python313\python.exe D:\course\软件安全\Lab4\model.py
Using cpu device
Train Epoch: 1  Loss: 0.286310
Test Accuracy = 9648 / 10000 = 0.9648
Train Epoch: 2  Loss: 0.052718
Test Accuracy = 9856 / 10000 = 0.9856
Train Epoch: 3  Loss: 0.050074
Test Accuracy = 9918 / 10000 = 0.9918
Train Epoch: 4  Loss: 0.015346
Test Accuracy = 9933 / 10000 = 0.9933
Train Epoch: 5  Loss: 0.098621
Test Accuracy = 9958 / 10000 = 0.9958
```

准确率大于 99%，配置正常。

首先需要实现对抗样本攻击中的 Fast Gradient Sign Attack(FGSM)。FGSM 的思想是在输入图片增加一些扰动，使得模型对于图片的预测发生错误。

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

1.1 untargeted attack

实现 untargeted Fast Gradient Sign Attack，即只需要让模型对图片的预测发生错误即可，不需要关注预测的结果。模型使用 model.py 里的模型结构，数据使用 /data 中的，首先先用 data 训练模型，保存在 mnist_model.pth 结果和测试配置时一样。实现 FGSM 攻击的核心代码如下：

```
def fgsm_step(model, data, label, eps, targeted=False, perturb_mask=None):
    adv_data = data.detach().clone()
    adv_data.requires_grad_(True)
    output = model(adv_data)
    loss = F.nll_loss(output, label)
    model.zero_grad()
    loss.backward()
    signed_grad = adv_data.grad.sign()

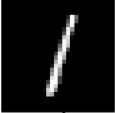
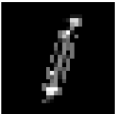
    if perturb_mask is not None:
        signed_grad = signed_grad * perturb_mask

    if targeted:
        adv_data = adv_data - eps * signed_grad
    else:
        adv_data = adv_data + eps * signed_grad

    return adv_data.detach().clamp(0, 1)
```

根据 FGSM 的理论公式生成上述代码，输入图片、扰动大小和梯度，首先先取梯度符号确定施加扰动的方向，再根据公式，以一定参数生成对抗样本，最后裁剪像素值限制大小在 $[0, 1]$ 内，确保生成的图像是有效图像，不影响人眼辨识。采用不同 epsilon 来测试准确率变化情况，生成可视化样本和数据结果如下，采用多部攻击的方法来让效果更好：

`step 8: success/total=1955/2000, success_rate=0.9775
saved examples: outputs\task32_targeted\task32_step08.png`

 y=7 pred=7	 y=2 pred=2	 y=1 pred=1	 y=0 pred=0	 y=4 pred=4	 y=9 pred=9	 y=5 pred=5	 y=6 pred=6
 y=7 pred=3	 y=2 pred=0	 y=1 pred=5	 y=0 pred=6	 y=4 pred=9	 y=9 pred=2	 y=5 pred=6	 y=6 pred=0

可以看到满足了基本要求：在 $\epsilon = 0.05$ 时成功率超过 95%。同时检查增加扰动后的图片可以看出，还是有点人眼辨识，但模型识别的正确率大幅度下降。

1.2 targeted attack

实现 targeted Fast Gradient Sign Attack，即需要让模型对图片的预测结果为一个固定值。例如对于所有预测结果不是1的图片，无论其标签是什么，都通过在上面加扰动的方式让模型的预测结果变成 1。在 Task1.1 的基础上修改代码，1.1 的逻辑是自由增加扰动，让图片朝着非原本预测的方向发展，那么为了定向攻击，需要减少梯度，设置损失函数让图片 1 为目标类型，统计时也改为统计为 1 的正确结果，只攻击原本预测正确的样本。

Targeted FGSM 与 untargeted FGSM 使用相同的梯度计算函数，但目标不同。这里希望模型把所有输入都预测为目标类别 1，因此先构造目标标签张量。当 targeted=True 时，fgsm_step 中使用的更新方向为减少方向，使用减号。

数据结果如下：

`step 9: fooled/base=1848/1929, error_rate=0.9580, remaining_accuracy=0.0420
saved examples: outputs\task31_untargeted\task31_step09.png`

 y=1 pred=1	 y=7 pred=7	 y=3 pred=3	 y=8 pred=8	 y=9 pred=9	 y=4 pred=4	 y=5 pred=3	 y=2 pred=2
 y=1 pred=1	 y=7 pred=1	 y=3 pred=1	 y=8 pred=1	 y=9 pred=1	 y=4 pred=1	 y=5 pred=1	 y=2 pred=1

可以看到满足了基本要求：在 $\epsilon = 0.05$ 时成功率超过 95%。同时检查增加扰动后的图片可以看出，大部分被导向识别成了 1。

task 2 后门攻击

实现后门攻击。后门攻击就是在模型训练时，在一部分训练数据上增加一个特定的 trigger。在训练时，对于这些带有 trigger 的输入，让模型将它们映射到某一个具体结果。例如，在一部分的图片的右下角加一个小方框作为 trigger，对于这些有 trigger 的图片，训练时让它们都映射到 1。这样当使用模型进行预测时，只要往图片右下角加入相同的小方框，无论该图片的标签实际是多少，都能让模型对该图片的预测结果为 1。

trigger 代码如下：

```
def add_natural_trigger(data):
    triggered = data.clone()
    coords = [
        (22, 24, 0.85),
        (23, 23, 0.95),
        (24, 22, 0.95),
        (25, 21, 0.85),
        (26, 20, 0.65),
        (23, 24, 0.45),
        (24, 23, 0.45),
        (25, 22, 0.45),
    ]
    for row, col, value in coords:
        triggered[..., row, col] = torch.maximum(
            triggered[..., row, col],
            torch.as_tensor(value, dtype=triggered.dtype,
                             device=triggered.device)
        )
    return triggered.clamp(0, 1)
```

因为以小方块作为 trigger 比较显眼，因此此处使用一道像素。

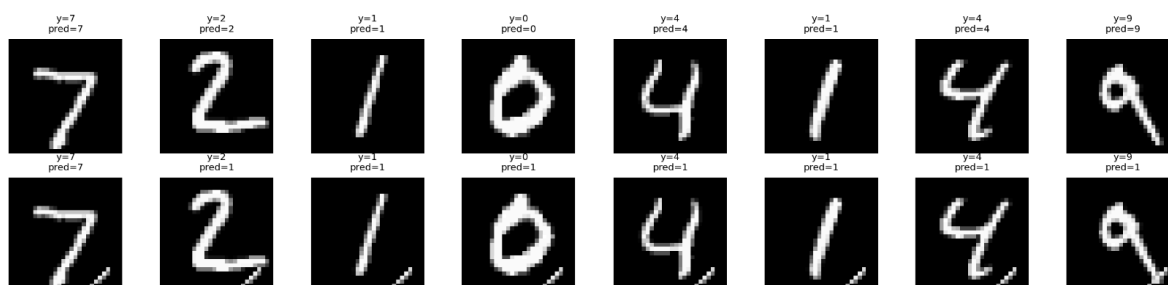
```
class BackdoorDataset(Dataset):
    def __init__(self, dataset, poison_rate=0.01, target_label=1, seed=42):
        self.dataset = dataset
        self.target_label = target_label
        labels = np.array([int(dataset[index][1]) for index in
                           range(len(dataset))])
        candidates = np.where(labels != target_label)[0]
        poison_count = max(1, int(len(dataset) * poison_rate))
        rng = np.random.default_rng(seed)
        self.poison_indices = set(rng.choice(candidates, size=poison_count,
                                              replace=False).tolist())

    def __getitem__(self, index):
        data, label = self.dataset[index]
        if index in self.poison_indices:
            data = add_natural_trigger(data)
            label = self.target_label
        return data, label
```

这段代码随机选择训练集中 $\eta=0.01$ 的样本进行投毒。对于被选中的样本，先加入 trigger，再把标签改成目标标签 1。这样模型训练时会学习到 trigger 与目标标签 1 的关联。测试时对所有测试图片添加同样的 trigger，如果模型预测为目标标签 1，就认为后门攻击成功。

数据结果如下：

```
Backdoor attack: eta=0.01, target=1
Poisoned samples = 200
Train Epoch: 1 Loss: 0.187972
Test Accuracy = 1874 / 2000 = 0.9370
Backdoor ASR = 0.9605
epoch 1: clean_accuracy=0.9370, backdoor_asr=0.9605
saved examples: outputs\task4_backdoor\task4_epoch01.png
```



可以看到满足了基本要求：在 $\epsilon = 0.05$ 时成功率超过 95%。同时检查增加扰动后的图片可以看出，大部分被导向识别成了 1。